

Question

Write a Java program to demonstrate multithreading and synchronized seat reservation.

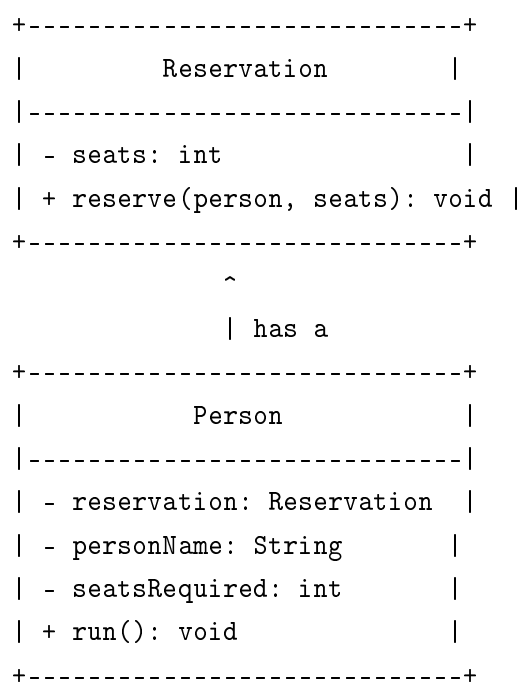
Steps for Execution

1. Save the code as `MultiThreading.java`.
2. Compile the program.
3. Run `MultiThreading`.

Algorithm

1. Create a reservation with 100 seats.
2. Create four person threads with different seat requests.
3. Start all threads.
4. Use the synchronized method to let one thread reserve seats at a time.
5. Display each booking result and the remaining seats.

UML Class Diagram



```

      ^
      | extends
+-----+
|      java.lang.Thread      |
+-----+

+-----+
|      MultiThreading        |
|-----|
| + main(args: String[]): void|
+-----+

```

CODE: SRC

MultiThreading.java

```

class Reservation {

    private int seats = 100;

    // Synchronized method
    synchronized void reserve(String person, int seatsRequired) {

        System.out.println(person + " is trying to book "
                           + seatsRequired + " seats.");

        if (seatsRequired <= seats) {

            System.out.println(person + " is booking seats...");

            seats = seats - seatsRequired;

            System.out.println(person + " successfully booked "
                               + seatsRequired + " seats.");

            System.out.println("Seats remaining: " + seats);

        } else {

            System.out.println(person
                               + " - Not enough seats available.");

        }

    }
}

```

```

        System.out.println();
    }
}

class Person extends Thread {

    Reservation reservation;
    String personName;
    int seatsRequired;

    Person(Reservation reservation, String personName,
           int seatsRequired) {

        this.reservation = reservation;
        this.personName = personName;
        this.seatsRequired = seatsRequired;
    }

    public void run() {
        reservation.reserve(personName, seatsRequired);
    }
}

public class MultiThreading {

    public static void main(String[] args) {

        Reservation reservation = new Reservation();

        Person p1 = new Person(reservation, "Person 1", 40);
        Person p2 = new Person(reservation, "Person 2", 30);
        Person p3 = new Person(reservation, "Person 3", 20);
        Person p4 = new Person(reservation, "Person 4", 25);

        p1.start();
        p2.start();
        p3.start();
        p4.start();
    }
}

```

Compilation and Execution

```
javac MultiThreading.java
java MultiThreading
```

Output

One possible output is shown below. Thread order can vary.

```
Person 1 is trying to book 40 seats.
Person 1 is booking seats...
Person 1 successfully booked 40 seats.
Seats remaining: 60
```

```
Person 2 is trying to book 30 seats.
Person 2 is booking seats...
Person 2 successfully booked 30 seats.
Seats remaining: 30
```

```
Person 3 is trying to book 20 seats.
Person 3 is booking seats...
Person 3 successfully booked 20 seats.
Seats remaining: 10
```

```
Person 4 is trying to book 25 seats.
Person 4 - Not enough seats available.
```